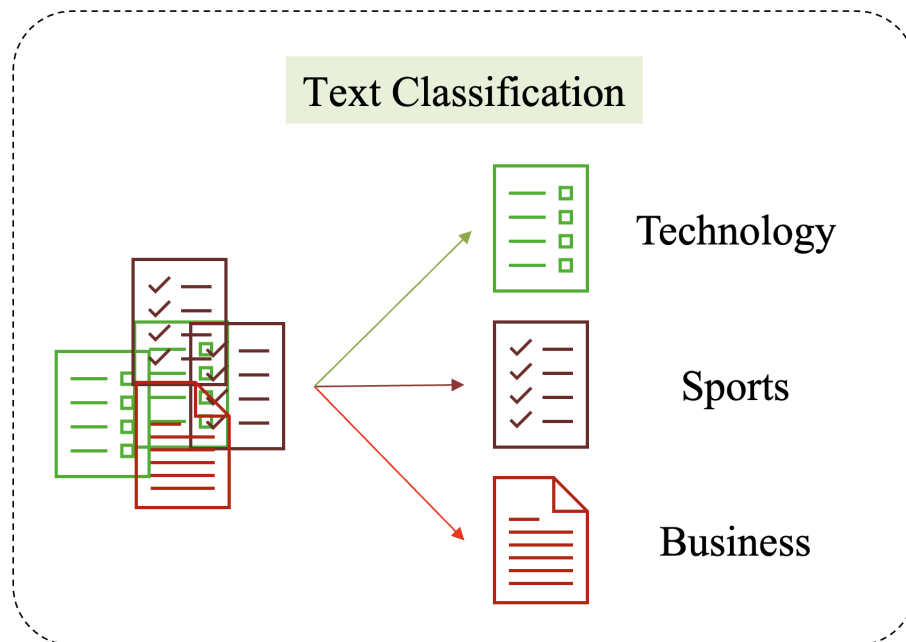


Sentiment Analysis - Project

Ngày 13 tháng 9 năm 2024

Phần I: Giới thiệu

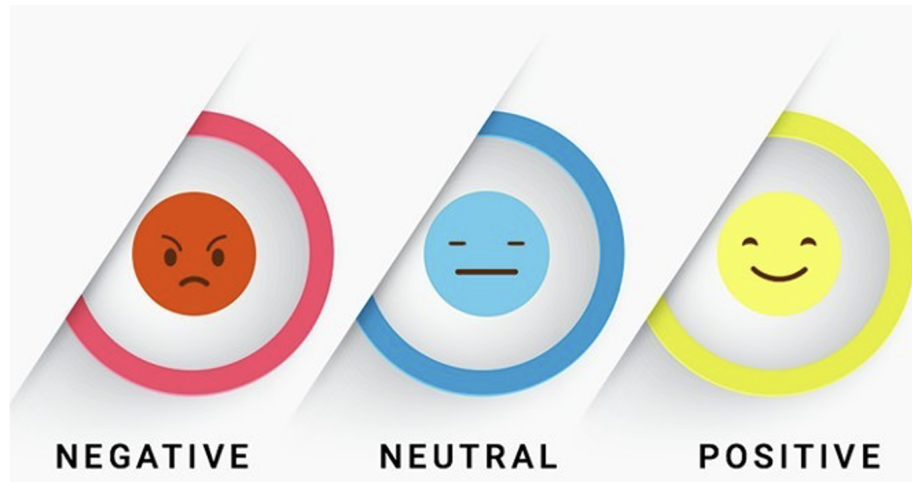
Trong bài tập này, chúng ta sẽ thực hành các nội dung về phân tích cảm xúc của khách hàng với các bình luận đánh giá phim dựa vào các phương pháp tiếp cận cho bài toán phân loại văn bản điển hình. Bài toán phân loại được lấy ví dụ như hình dưới. Với mỗi đơn vị văn bản sẽ được phân loại với một nhãn cụ thể thuộc tập hợp các nhãn cho trước.



Hình 1: Text Classification.

Với mỗi đơn vị văn bản có thể thuộc vào tập các nhãn như: 'Technology', 'Sports', 'Business',...

Sentiment Analysis là nhóm các bài toán con thuộc vào phân loại văn bản. Với mục tiêu phân tích và đánh giá các bình luận của khách hàng cho các sản phẩm và tích cực, tiêu cực hay trung tính.



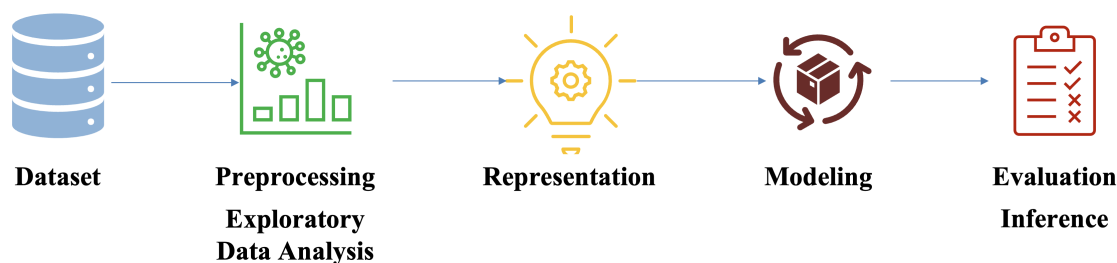
Hình 2: Sentiment Analysis.

Phần project này sẽ tập trung giải quyết cho bài phân tích cảm xúc trên bộ dữ liệu IMDB - Đánh giá phim.

Phần II: Bài tập

A. Phần lập trình

Trong phần này, chúng ta sẽ cài đặt và huấn luyện hai mô hình Decision Tree và Random Forest để giải quyết bài toán phân tích cảm xúc gồm các bước như hình sau:



Hình 3: Các bước huấn luyện mô hình phân loại.

1. **Tải bộ dữ liệu:** Các bạn tải bộ dữ liệu IMDB-Dataset.csv tại [đây](#).
2. **Đọc bộ dữ liệu:** Sử dụng thư viện pandas, chúng ta sẽ đọc file .csv lên như sau:

```
1 # Load dataset
2 import pandas as pd
3
4 df = pd.read_csv('./IMDB-Dataset.csv')
5
6 # Remove duplicate rows
7 df = df.drop_duplicates()
```

Ở đây chúng ta sẽ thực hiện làm sạch dữ liệu thông qua các bước như: xoá thẻ html, xoá dấu câu, xoá số, xoá các icon,...

```
1 import re
2 import string
3 import nltk
4 nltk.download('stopwords')
5 nltk.download('wordnet')
6 from nltk.corpus import stopwords
7 from nltk.stem import WordNetLemmatizer
8 from bs4 import BeautifulSoup
9 import contractions
10
11 stop = set(stopwords.words('english'))
12
13 # Expanding contractions
14 def expand_contractions(text):
15     return contractions.fix(text)
16
17 # Function to clean data
18 def preprocess_text(text):
19
20     wl = WordNetLemmatizer()
21
22     soup = BeautifulSoup(text, "html.parser") # Removing html tags
```

```

23 text = soup.get_text()
24 text = expand_contractions(text) # Expanding chatwords and contracts clearing
    contractions
25 emoji_clean = re.compile("[
26         u"\U0001F600-\U0001F64F" # emoticons
27         u"\U0001F300-\U0001F5FF" # symbols & pictographs
28         u"\U0001F680-\U0001F6FF" # transport & map symbols
29         u"\U0001F1E0-\U0001F1FF" # flags (iOS)
30         u"\U00002702-\U000027B0"
31         u"\U000024C2-\U0001F251"
32         "]" + "", flags=re.UNICODE)
33 text = emoji_clean.sub(r'', text)
34 text = re.sub(r'\.(?=\S)', '. ', text) #add space after full stop
35 text = re.sub(r'http\S+', '', text) #remove urls
36 text = " ".join([
37     word.lower() for word in text if word not in string.punctuation
38 ]) #remove punctuation and make text lowercase
39 text = " ".join([
40     wl.lemmatize(word) for word in text.split() if word not in stop and word.
    isalpha()]) #lemmatize
41 return text
42
43 df['review'] = df['review'].apply(preprocess_text)

```

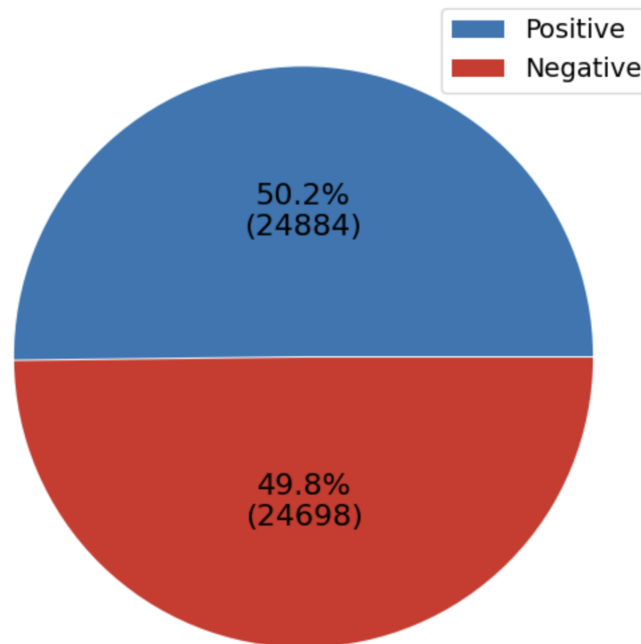
3. Phân tích dữ liệu: Thống kê số lượng các nhãn trong bộ dữ liệu:

```

1 import numpy as np
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # Creating autocpt arguments
6 def func(pct, allvalues):
7     absolute = int(pct / 100.*np.sum(allvalues))
8     return "{:.1f}%\n({:d})".format(pct, absolute)
9
10 freq_pos = len(df[df['sentiment'] == 'positive'])
11 freq_neg = len(df[df['sentiment'] == 'negative'])
12
13 data = [freq_pos, freq_neg]
14
15 labels = ['positive', 'negative']
16 # Create pie chart
17 pie, ax = plt.subplots(figsize=[11,7])
18 plt.pie(x=data, autopct=lambda pct: func(pct, data), explode=[0.0025]*2,
19     pctdistance=0.5, colors=[sns.color_palette()[0], 'tab:red'], textprops={'
20     fontsize': 16})
21 # plt.title('Frequencies of sentiment labels', fontsize=14, fontweight='bold')
22 labels = [r'Positive', r'Negative']
23 plt.legend(labels, loc="best", prop={'size': 14})
24 pie.savefig("PieChart.png")
25 plt.show()

```

Kết quả thu được:



Hình 4: Số lượng các nhãn trong bộ dữ liệu IMDB.

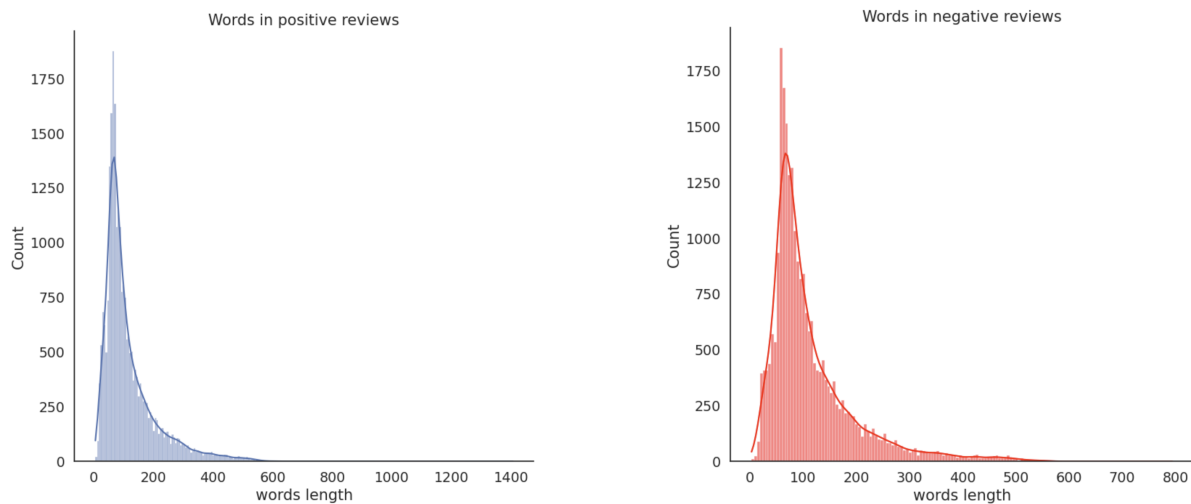
Thống kê độ dài của các mẫu cho mỗi class.

```

1 words_len = df['review'].str.split().map(lambda x: len(x))
2 df_temp = df.copy()
3 df_temp['words length'] = words_len
4
5 hist_positive = sns.displot(
6     data=df_temp[df_temp['sentiment'] == 'positive'],
7     x="words length", hue="sentiment", kde=True, height=7, aspect=1.1, legend=
8     False
9 ).set(title='Words in positive reviews')
10 plt.show(hist_positive)
11
12 hist_negative = sns.displot(
13     data=df_temp[df_temp['sentiment'] == 'negative'],
14     x="words length", hue="sentiment", kde=True, height=7, aspect=1.1, legend=
15     False, palette=['red']
16 ).set(title='Words in negative reviews')
17 plt.show(hist_negative)
18
19 plt.figure(figsize=(7,7.1))
20 kernel_distribution_number_words_plot = sns.kdeplot(
21     data=df_temp, x="words length", hue="sentiment", fill=True, palette=[sns.
22     color_palette()[0], 'red']
23 ).set(title='Words in reviews')
24 plt.legend(title='Sentiment', labels=['negative', 'positive'])
25 plt.show(kernel_distribution_number_words_plot)

```

Kết quả thu được:



Hình 5: Số lượng các nhãn trong bộ dữ liệu IMDB.

4. Chia tập train và test:

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.feature_extraction.text import TfidfVectorizer
3 from sklearn.preprocessing import LabelEncoder
4
5 label_encode = LabelEncoder()
6 y_data = label_encode.fit_transform(df['sentiment'])
7
8 x_train, x_test, y_train, y_test = train_test_split(
9     x_data, y_data, test_size=0.2, random_state=42
10 )
```

5. Biểu diễn văn bản thành vector:

```
1 tfidf_vectorizer = TfidfVectorizer(max_features=10000)
2 tfidf_vectorizer.fit(x_train, y_train)
3
4 x_train_encoded = tfidf_vectorizer.transform(x_train)
5 x_test_encoded = tfidf_vectorizer.transform(x_test)
```

6. Huấn luyện và đánh giá mô hình:

Ta thực hiện huấn luyện mô hình với bộ dữ liệu train. Để huấn luyện mô hình Decision Tree, các bạn sẽ sử dụng DecisionTreeClassifier():

```
1 from sklearn.tree import DecisionTreeClassifier
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.metrics import accuracy_score
4
5 dt_classifier = DecisionTreeClassifier(
6     criterion='entropy',
7     random_state=42
8 )
9 dt_classifier.fit(x_train_encoded, y_train)
10 y_pred = dt_classifier.predict(x_test_encoded)
11 accuracy_score(y_pred, y_test)
```

Để huấn luyện mô hình Random Forest, các bạn sẽ sử dụng RandomForestClassifier():

```
1 rf_classifier = RandomForestClassifier(  
2     random_state=42  
3 )  
4 rf_classifier.fit(x_train_encoded, y_train)  
5 y_pred = rf_classifier.predict(x_test_encoded)  
6 accuracy_score(y_pred, y_test)
```

Bên cạnh các mô hình như Decision Tree hoặc Random Forest, chúng ta có thể mở rộng đánh giá và so sánh với các mô hình học máy khác như AdaBoost, Gradient Boosting và XGBoost. Từ đó xác định xem các mô hình nào sẽ hiệu quả và có độ chính xác cao trên bài toán phân loại với bộ dữ liệu IMDB.

B. Phần trắc nghiệm

1. Độ đo đánh giá mô hình cho bài toán phân loại văn bản là?
(a) **Accuracy** (c) ROUGE
(b) BLEU (d) COMET
2. Bước nào không thuộc vào mô hình phân loại ở phần 2?
(a) Modeling (c) Text Representation
(b) Text Preprocessing (d) **Data Crawling**
3. Mô hình nào có thể được sử dụng làm bộ phân loại cho bài toán phân loại văn bản?
(a) Decision Tree Classifier (c) **TF-IDF**
(b) Decision Tree Regressor (d) BoW
4. Phương pháp biểu diễn văn bản thành vector được sử dụng trong phần thực nghiệm là?
(a) One-hot Encoding (c) **TF-IDF**
(b) Bag Of Word (d) Bag Of N-gram
5. Bộ dữ liệu được sử dụng cho bài toán phân loại trong phần thực nghiệm là?
(a) **IMDB** (c) Play Tennis
(b) IRIS (d) OPUS
6. Số lượng sample trùng lặp trong bộ dữ liệu phân loại được sử dụng trong phần thực nghiệm là?
(a) 418 (c) 518
(b) 318 (d) 218
7. Kích thước bộ từ điển sau bước tiền xử lý trong phần thực nghiệm xấp xỉ với kết quả nào sau đây?
(a) **100000 từ** (c) 140000 từ
(b) 120000 từ (d) 160000 từ
8. Khai báo nào sau đây là đúng?
(a) **TfidfVectorizer(max_features=10000)** (c) TfidfVectorizer(max_feature=10000)
(b) TfidfVector(max_features=10000) (d) TfidfVector(max_features=10000)

9. Kết quả độ chính xác accuracy của mô hình Random Forest Classifier trên tập xấp xỉ với kết quả nào sau đây?

(a) 64

(c) 84

(b) 74

(d) 94

10. Lớp LabelEncoder có hàm nào sau đây?

(a) inverse_transform()

(c) fit_transforms()

(b) inverse_transforms()

(d) fit_data()

- **Hết** -